

# Big Data Analytics Pipeline Using Apache Spark, Hadoop HDFS, Apache Hive and Apache Superset

Beyzanur Arslan

July 2026

## 1 Introduction

The increasing volume of data generated by e-commerce platforms has made big data technologies essential for data storage, processing, and analysis. Organizations use distributed systems to transform raw transactional data into meaningful business insights that support operational and strategic decision-making.

This project implements an end-to-end big data analytics pipeline using Apache Spark, Hadoop HDFS, Apache Hive, and Apache Superset. The publicly available Olist Brazilian E-Commerce dataset was processed by converting raw CSV files into Parquet format, storing them in Hadoop HDFS, creating Hive external tables, and visualizing the data through an interactive dashboard in Apache Superset.

The main objective of this project is to demonstrate how modern big data tools can be integrated into a complete analytics workflow while providing useful business insights through interactive visualizations.

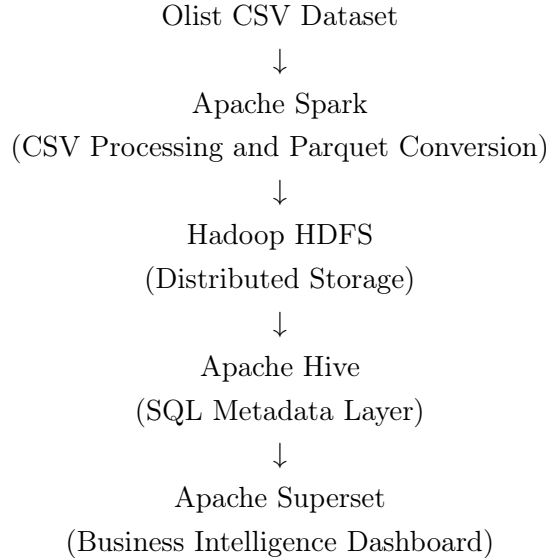
## 2 Project Objectives

The project was designed with the following objectives:

- Import multiple CSV datasets using Apache Spark.
- Transform raw CSV files into the Parquet format.
- Store processed datasets in Hadoop HDFS.
- Create Hive external tables for SQL querying.
- Connect Apache Superset to Hive.
- Develop an interactive dashboard containing key business metrics.
- Demonstrate an end-to-end big data analytics pipeline.

### 3 System Architecture

The overall architecture follows a layered big data design where each technology performs a dedicated task within the pipeline.



The pipeline begins with raw CSV files obtained from the Olist dataset. Apache Spark reads every dataset, automatically infers the schema, and converts each table into the Parquet format. Compared to CSV files, Parquet significantly improves analytical query performance because it stores data in a compressed columnar format.

After transformation, the Parquet datasets are uploaded to Hadoop HDFS. HDFS provides reliable distributed storage and allows the datasets to be accessed by multiple services within the analytics environment.

Apache Hive is then configured to create external tables that reference the Parquet files stored inside HDFS. This approach avoids unnecessary data duplication while enabling SQL queries over the distributed storage system.

Finally, Apache Superset connects directly to Hive and provides an interactive interface for dashboard creation and business analysis.

### 4 Data Processing Pipeline

The original dataset consists of nine CSV files containing information about customers, orders, products, sellers, payments, reviews, and geographical locations.

Apache Spark was selected as the processing engine due to its distributed computing capabilities and efficient support for large-scale data transformation. During the ingestion phase, Spark automatically inferred the schema of each dataset and loaded the CSV files into DataFrames.

Once the datasets were successfully loaded, every table was converted into the Apache Parquet format. This conversion provides several important advantages:

- Faster analytical queries

- Reduced storage requirements through compression
- Column-oriented storage
- Better compatibility with distributed query engines

After the transformation stage, the generated Parquet files were stored inside Hadoop HDFS. Hive external tables were then created on top of these files, allowing SQL-based access without copying the underlying data.

This architecture separates computation, storage, and visualization into independent layers, making the pipeline scalable and easier to maintain.

## 5 Dashboard Analysis

After completing the data processing pipeline, Apache Superset was used to create an interactive dashboard that summarizes the most important business metrics. The dashboard provides insights into customer behavior, sales performance, seller activity, delivery efficiency, and regional demand. These visualizations allow users to quickly understand key trends and support data-driven decision-making.

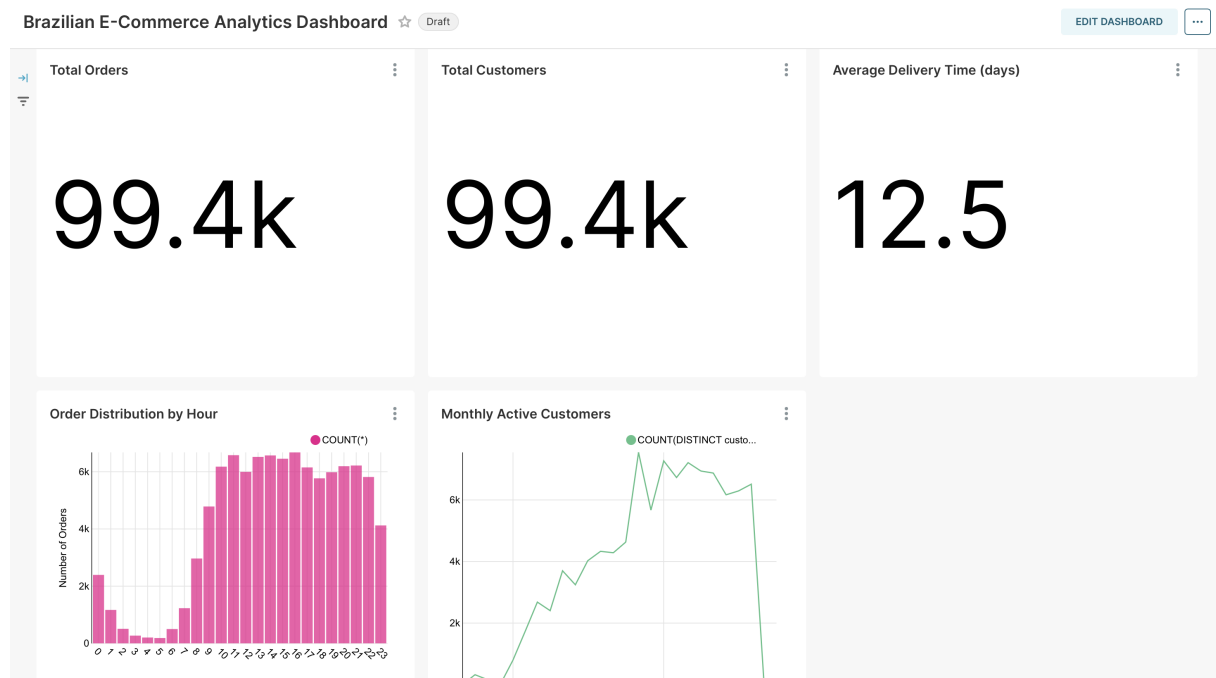


Figure 1: Overall Apache Superset dashboard showing key business metrics.

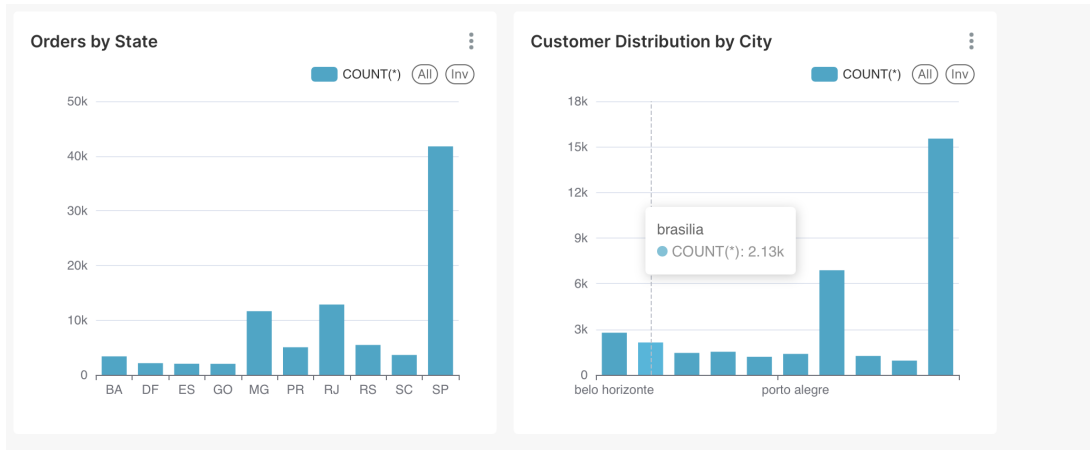


Figure 2: Sales and customer analytics, including monthly activity and order status distribution.

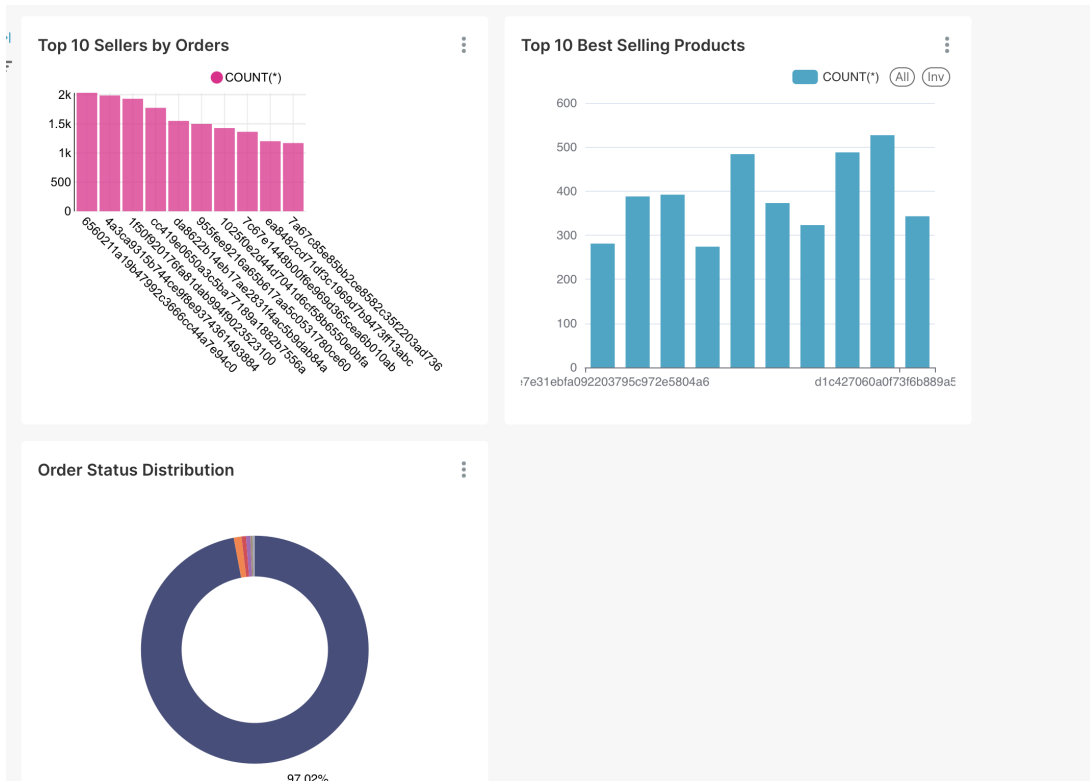


Figure 3: Geographical distribution of customers and seller performance analysis.

## 5.1 Total Orders

The Total Orders KPI displays the overall number of customer orders contained in the dataset. This metric represents the overall business volume of the marketplace and serves as a reference point for evaluating all other business indicators. Monitoring total orders over time enables companies to measure growth, compare different reporting periods, and estimate future demand.

## 5.2 Total Customers

The Total Customers KPI measures the number of unique customers who placed orders during the observation period. Customer acquisition is one of the most important indicators for e-commerce businesses because sustainable growth depends not only on increasing sales but also on expanding the customer base.

Comparing the number of customers with the total number of orders also provides an indication of customer purchasing behavior.

## 5.3 Average Delivery Time

Delivery performance is one of the most critical factors affecting customer satisfaction in online shopping.

This visualization calculates the average delivery duration between purchase and successful delivery. Companies continuously monitor this metric to evaluate logistics performance, identify delivery delays, and improve warehouse operations.

Lower delivery times generally contribute to higher customer satisfaction and stronger marketplace performance.

## 5.4 Monthly Active Customers

The Monthly Active Customers line chart illustrates how customer activity changes over time.

Instead of examining a single summary value, this visualization allows analysts to identify seasonal trends, periods of rapid growth, and fluctuations in customer engagement.

Understanding customer activity patterns supports marketing campaigns, demand forecasting, and long-term business planning.

## 5.5 Order Status Distribution

This chart presents the distribution of order statuses within the marketplace.

Most successful e-commerce platforms aim for a high percentage of delivered orders while minimizing canceled or unavailable orders.

Monitoring order status distribution enables managers to detect operational issues, evaluate fulfillment performance, and improve customer service processes.

## 5.6 Orders by State

Customer demand is not distributed equally across different regions.

The Orders by State visualization groups all orders according to Brazilian states, allowing businesses to identify regions with the highest sales activity.

Such geographic analysis supports warehouse planning, regional marketing campaigns, inventory allocation, and logistics optimization.

## 5.7 Customer Distribution by City

Unlike state-level analysis, this chart focuses on individual cities.

The visualization identifies cities containing the largest customer populations and highlights areas where business demand is concentrated.

Companies frequently use this information when planning regional promotions, delivery centers, and expansion strategies.

## 5.8 Top 10 Best Selling Products

This visualization ranks the products with the highest number of completed sales.

Best-selling products represent the core revenue generators of an e-commerce platform. Understanding product popularity enables businesses to optimize inventory levels, improve supplier relationships, and prioritize marketing efforts.

The chart also provides valuable information for forecasting future demand.

## 5.9 Top 10 Sellers by Orders

Marketplace platforms depend heavily on seller performance.

This visualization ranks sellers according to the number of orders they successfully fulfilled.

High-performing sellers contribute significantly to marketplace activity, while low-performing sellers may require operational support or performance evaluation.

Such analysis assists marketplace administrators in monitoring seller productivity.

## 5.10 Order Distribution by Hour

Customer purchasing behavior varies considerably throughout the day.

This visualization groups orders according to purchase hour, making it possible to identify peak shopping periods.

Knowing when customers are most active allows businesses to optimize advertising schedules, customer support availability, website infrastructure, and promotional campaigns.

Overall, the dashboard transforms transactional data into actionable business information that can support both operational monitoring and strategic decision-making.

# 6 Discussion

The implemented analytics pipeline demonstrates how multiple open-source technologies can be integrated into a complete business intelligence workflow.

Apache Spark efficiently processed multiple large datasets and transformed them into the Parquet format, significantly improving storage efficiency and analytical performance.

Hadoop HDFS provided a scalable distributed storage environment, while Hive simplified SQL-based data access without duplicating the stored files.

Apache Superset successfully converted the processed datasets into interactive visualizations, allowing business users to explore operational metrics without writing SQL queries.

Although the project focuses primarily on descriptive analytics, the implemented architecture establishes a solid foundation for future machine learning applications, predictive analytics, recommendation systems, and customer segmentation models.

## 7 Phase 2 - Data Warehouse Design and Business Analytics

This phase extends the previously developed data pipeline by focusing on data quality, warehouse design, ETL architecture, and business-oriented analytics. The objective is to transform raw e-commerce data into an analytical model that supports business decision-making through SQL queries and interactive dashboards.

### 7.1 Data Quality Assessment

Before building the analytical model, the input datasets were examined to evaluate their overall quality. Clean data refers to data that is accurate, complete, consistent, and free from duplicate or invalid records.

The Olist dataset is generally well-structured; however, several preprocessing steps are required before analysis. Missing values, inconsistent data types, and duplicate entities should be identified during the transformation process. In this project, Spark automatically inferred the schema of each dataset, while unnecessary columns were removed and appropriate joins were applied during fact table creation.

Each table contains a primary identifier such as `order_id`, `customer_id`, `seller_id`, or `product_id`. These identifiers were used during joins to avoid duplicate records in the analytical dataset. The final fact table contains one record for each order item together with payment, seller, customer, and review information.

### 7.2 Output Dataset Design

The raw Olist dataset consists of multiple normalized tables. Although this structure is suitable for transactional systems, it is not efficient for business analytics because multiple joins are required for almost every query.

To simplify reporting, a consolidated analytical dataset named `fact_orders` was created. This table combines the most important business attributes including:

- Order information
- Customer information
- Seller information
- Product information
- Payment details
- Review score
- Order status

This denormalized structure enables faster aggregation and simplifies dashboard development in Apache Superset.

### 7.3 ETL Approach

An ETL (Extract, Transform, Load) architecture was adopted throughout this project.

During the Extract phase, raw CSV files were loaded into Apache Spark DataFrames.

During the Transform phase, the datasets were cleaned, joined, converted into Parquet format, and reorganized into a business-oriented analytical structure.

Finally, during the Load phase, the transformed Parquet files were stored in Hadoop HDFS and exposed through Apache Hive external tables for SQL querying.

The ETL approach was selected because the transformations were completed before loading the data into the analytical environment, resulting in faster dashboard queries and improved reporting performance.

### 7.4 ETL vs ELT and dbt

Although both ETL and ELT are widely used in modern data engineering, ETL was more appropriate for this project because Apache Spark provides powerful distributed processing capabilities before data is loaded into Hive.

In an ELT architecture, raw data is first loaded into the warehouse and transformed afterwards using SQL. This approach is common in cloud-based platforms such as Snowflake and BigQuery.

Another popular transformation tool is dbt (Data Build Tool). dbt performs SQL-based transformations inside the data warehouse while supporting modular models, testing, documentation, and version control. Although dbt was not used in this implementation, it would be a suitable solution for managing large-scale analytical projects.

### 7.5 Architecture Layers

The implemented architecture consists of five logical layers.

1. Data Source Layer (CSV files)
2. Processing Layer (Apache Spark)
3. Storage Layer (Hadoop HDFS)
4. Query Layer (Apache Hive)
5. Visualization Layer (Apache Superset)

Each layer has a dedicated responsibility. Spark performs distributed data processing, HDFS stores the transformed datasets, Hive provides SQL access through metadata, and Superset generates interactive business dashboards.

### 7.6 Star Schema Design

To support analytical queries, a star schema was designed around a central fact table.

The central fact table stores transactional measures including payment amount, product price, freight value, review score, and payment installments.



Dimension tables describe the surrounding business entities such as customers, sellers, products, payment methods, dates, and order status.

This structure minimizes query complexity while improving aggregation performance for business intelligence applications.

## 7.7 Fact and Dimension Tables

The primary analytical table developed in this project is **fact\_orders**. It stores the numerical business measures that are aggregated during analysis.

The main dimensions include:

- Customer Dimension
- Seller Dimension
- Product Dimension
- Payment Dimension
- Date Dimension
- Order Status Dimension

The separation of facts and dimensions enables flexible reporting across different business perspectives while avoiding unnecessary data duplication.

## 7.8 Business Questions and Analytical Model

The developed warehouse design supports several common business questions.

| Business Question      | Fact Table  | Main Dimensions |
|------------------------|-------------|-----------------|
| Monthly Revenue        | fact_orders | Date            |
| Top-performing Sellers | fact_orders | Seller          |
| Payment Method Trends  | fact_orders | Payment Type    |
| Order Status Analysis  | fact_orders | Order Status    |
| Customer Distribution  | fact_orders | Customer        |
| Average Review Score   | fact_orders | Review          |

Table 1: Business questions supported by the analytical model.

The star schema allows these analytical queries to be executed efficiently while keeping the SQL logic simple.

## 7.9 Dashboard Visualizations

Apache Superset was used to transform the analytical dataset into interactive visualizations.

The final dashboard contains several KPI cards together with business charts that summarize sales performance and customer behavior.

The implemented visualizations include:

- Total Sales
- Total Orders
- Total Customers
- Average Review Score
- Payment Type Distribution
- Order Status Distribution
- Revenue by Order Status
- Review Score Distribution

These visualizations provide an intuitive overview of marketplace performance. KPI cards summarize the overall business volume, while the charts reveal payment preferences, order completion rates, revenue contribution by status, and customer satisfaction through review scores.

The dashboard enables business users to monitor marketplace performance without writing SQL queries and supports data-driven decision making.

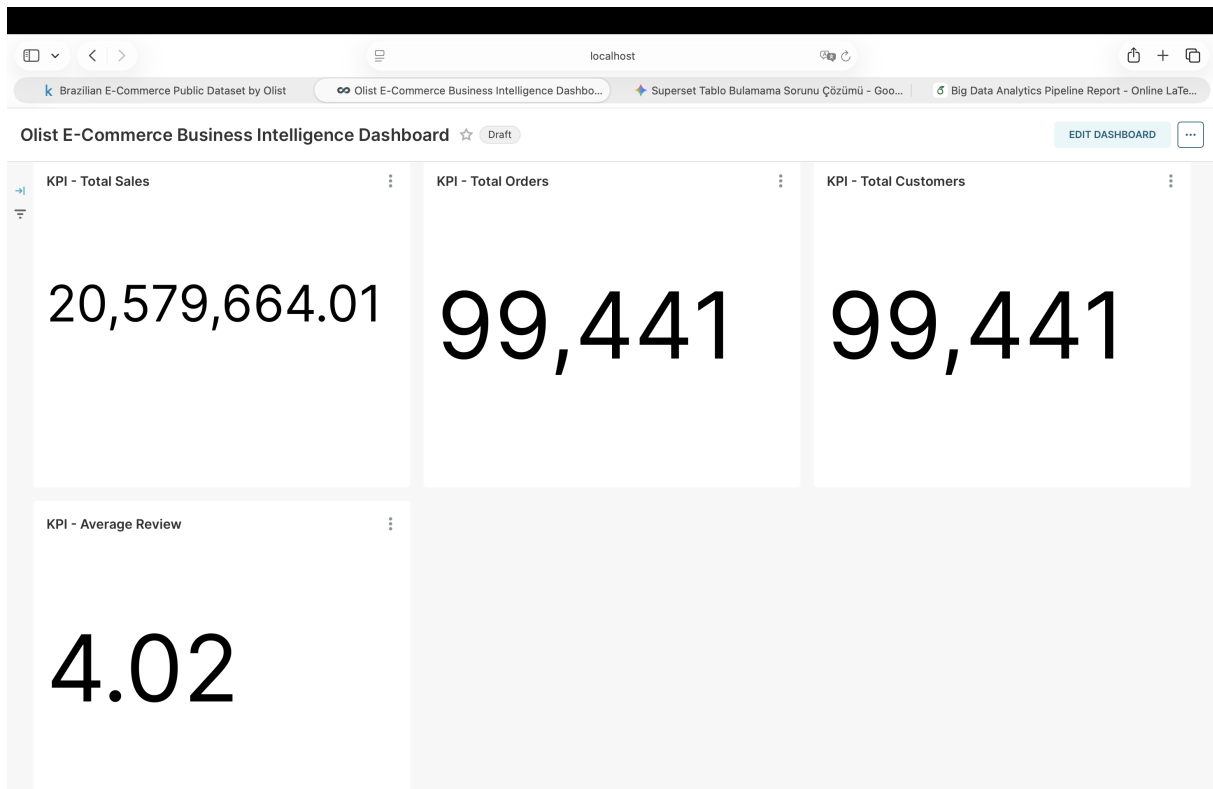


Figure 4: Apache Superset dashboard showing KPIs and analytical charts.

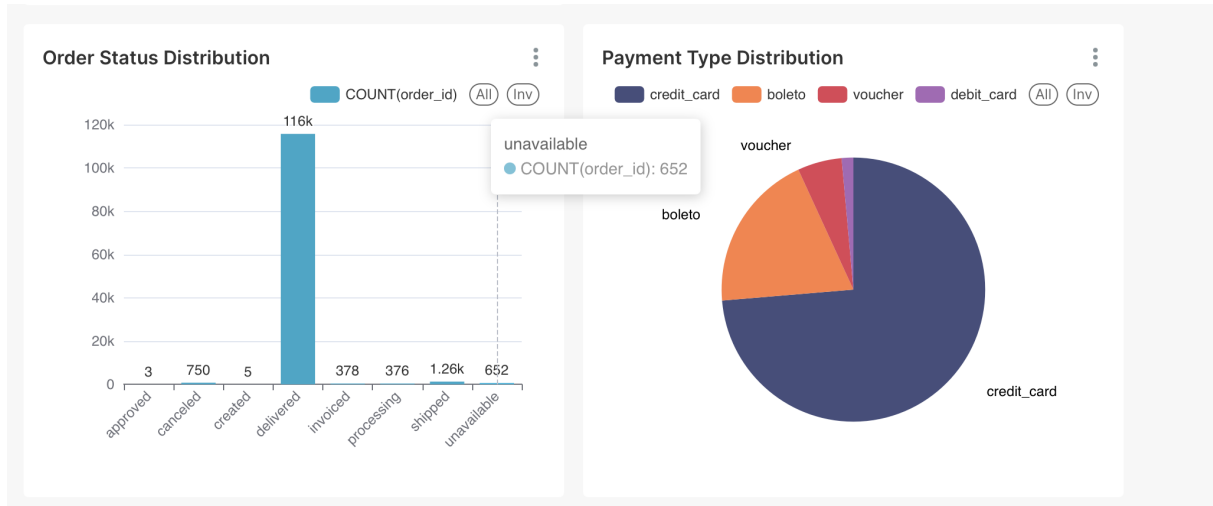


Figure 5: Revenue, payment type and order status analysis.

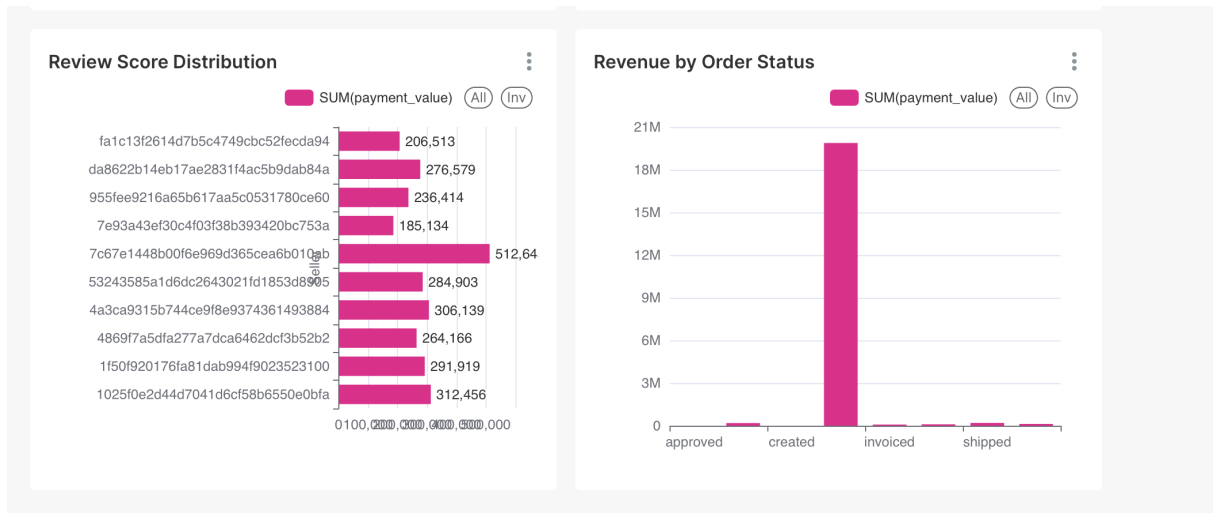


Figure 6: Customer review analysis and seller performance visualizations.

## 8 Conclusion

This project successfully implemented a complete end-to-end big data analytics solution using Apache Spark, Hadoop HDFS, Apache Hive, and Apache Superset.

In the first phase, raw CSV datasets were transformed into Parquet format, stored in HDFS, exposed through Hive, and analyzed using interactive dashboards in Apache Superset.

In the second phase, the project was extended by evaluating data quality, designing an analytical warehouse model, explaining the ETL process, defining a star schema, and demonstrating how the analytical model supports common business questions.

The final dashboard combines KPI indicators and business visualizations that help monitor sales performance, customer behavior, payment preferences, order status, and customer satisfaction. The proposed architecture demonstrates how distributed data engineering technologies can be integrated into a scalable business intelligence solution.

Future improvements may include automated ETL pipelines, additional dimension tables, predictive analytics, customer segmentation, and real-time data processing.

## References

- [1] Apache Spark. <https://spark.apache.org>
- [2] Apache Hadoop. <https://hadoop.apache.org>
- [3] Apache Hive. <https://hive.apache.org>
- [4] Apache Superset. <https://superset.apache.org>
- [5] Olist Brazilian E-Commerce Dataset. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

## 9 Phase 3 - Pipeline Reconstruction with Airflow and dbt

In Phase 3, the existing data pipeline was reconstructed by integrating Apache Airflow for workflow orchestration and dbt for analytical transformations. The objective was to automate the complete pipeline and reorganize the transformation process into Bronze, Silver, and Gold layers based on the Medallion Architecture.

Apache Airflow was configured with a webserver, scheduler, triggerer, LocalExecutor, and PostgreSQL metadata database. The main DAG, named `olist_medallion_pipeline`, manages the pipeline tasks in the following order:

Source Validation → Spark Ingestion → Bronze Validation → dbt Bronze → dbt Silver → dbt Gold → Data Tests → Superset Refresh

The DAG uses Python-based tasks for lightweight validation and Bash-based tasks for dbt commands. Retries, execution timeouts, service-readiness checks, and failure logging were included to improve reliability. XCom is used only for small metadata values such as row counts and validation results.

The dbt project restructures the analytical pipeline into three layers:

- **Bronze Layer:** Preserves source-aligned data with minimal transformations and safe type casting.
- **Silver Layer:** Cleans, types, and deduplicates records while creating reusable business entities.
- **Gold Layer:** Rebuilds the star schema containing `fact_orders`, `dim_customers`, `dim_products`, `dim_sellers`, and `dim_date`.

The `fact_orders` table maintains one row per order item. Payments are aggregated at order level and proportionally allocated to each item using its price and freight value. This prevents revenue duplication when an order contains multiple products or payment records.

dbt tests were added to verify unique keys, non-null values, relationships, accepted order statuses, review score ranges, positive numerical values, and payment reconciliation. The main benefits of this architecture are modularity, lineage, easier testing, fault isolation, and separation between ingestion and analytical modeling.

The main challenges were connecting Airflow containers to Spark and HDFS, selecting a compatible dbt adapter for Spark ThriftServer, preserving the correct fact-table grain, managing Docker platform differences, and maintaining Hive metadata across service restarts.

Overall, Phase 3 transforms the original manually executed pipeline into a more automated, testable, and maintainable data engineering solution.